Buổi 7

Danh sách liên kết đơn

Học phần: Cấu trúc dữ liệu và Giải thuật

Nội dung bài học

1. Khái niệm danh sách liên kết

1.1 Định nghĩa danh sách liên kết
1.2 Cấu trúc nút và cấp phát bộ nhớ động

2. Các thao tác trên danh sách liên kết đơn

2.1 Khởi tạo, tạo nút mới và duyệt danh sách
2.2 Tìm kiếm trên danh sách liên kết đơn
2.3 Thêm phần tử vào đầu danh sách
2.4 Thêm phần tử vào cuối danh sách
2.5 Thêm phần tử vào sau một nút và thêm có thứ tự
2.6 Hủy phần tử đầu danh sách
2.7 Hủy phần tử sau một nút và hủy theo khóa
2.8 Hủy toàn bộ danh sách
2.9 Sắp xếp trên danh sách liên kết đơn
2.10 Ví dụ: đảo ngược danh sách liên kết đơn

3. Lỗi con trỏ và cải tiến

3.1 Các lỗi con trỏ thường gặp
3.2 Quản lý danh sách bằng hai con trỏ First và Last

4. Tổng kết và thực hành

4.1 Bài thực hành buổi 7 – Quản lý danh sách sinh viên bằng DSLK đơn
4.2 Câu hỏi và bài tập

1.1 Định nghĩa danh sách liên kết

Định nghĩa

Danh sách liên kết là danh sách mà các phần tử (gọi là **nút** – node) được lưu ở các vùng nhớ **không nhất thiết liên tiếp**; mỗi nút ngoài **dữ liệu** còn chứa **liên kết** (con trỏ) tới nút khác. Danh sách được quản lý bởi con trỏ tới **nút đầu** (First / SLList).

- **Danh sách liên kết đơn (Singly Linked List – SLL):** mỗi nút có **một** liên kết NextNode trỏ tới nút **kế tiếp**; nút cuối có NextNode = **NULL**.

```
SLList
  ●──→[10|●]──→[20|●]──→[18|●]──→[40|●]──→ NULL
        nút đầu                      nút cuối
```

- Các nút được **cấp phát động** khi cần (new) và **giải phóng** khi hủy (delete) ⇒ kích thước danh sách **không giới hạn trước**.

Tiêu chí	Danh sách đặc	DSLK đơn
Vị trí các phần tử	Liên tiếp	Rải rác
Kích thước	Cố định MaxLen	Tùy ý (theo bộ nhớ)
Truy cập phần tử thứ i	O(1)	**O(i)** – phải đi từ đầu
Thêm/hủy đầu	O(N) – dời phần tử	**O(1)**
Thêm/hủy sau nút đã biết	O(N)	**O(1)** – chỉ đổi con trỏ
Bộ nhớ mỗi phần tử	sizeof(T)	sizeof(T) + con trỏ (8 byte)
Tìm nhị phân	Được	Không hiệu quả

1.2 Cấu trúc nút và cấp phát bộ nhớ động

- **Cấu trúc dữ liệu (theo giáo trình):**

```
typedef struct SLL_Node
{  T Key;                    // thành phần dữ liệu
   SLL_Node *NextNode;       // liên kết tới nút kế
} SLL_OneNode;
typedef SLL_OneNode *SLL_Type;   // con trỏ tới nút
SLL_Type SLList;                 // quản lý danh sách
```

- Có thể thay `T Key` bằng nhiều trường (MaSV, HoTen, DTB…) hoặc một struct.
- **Cấp phát / giải phóng trong C/C++:**

C++	C
`p = new SLL_OneNode;`	`p = (SLL_Type)malloc(sizeof(SLL_OneNode));`
`delete p;`	`free(p);`

Ví dụ 7.1 – Hình ảnh bộ nhớ của danh sách (10, 20, 18)

Địa chỉ	Key	NextNode
5000 (nút 1)	10	7200
7200 (nút 2)	20	6100
6100 (nút 3)	18	NULL

- SLList = 5000. Các nút **không** nằm liên tiếp (5000 → 7200 → 6100).
- Truy cập "phần tử thứ 3" phải đi: SLList → nút 1 → nút 2 → nút 3.

- **Quy ước ký hiệu:** p->Key là dữ liệu của nút do p trỏ tới; p->NextNode là địa chỉ nút kế; p->NextNode->Key là dữ liệu của nút kế.

2.1 Khởi tạo, tạo nút mới và duyệt danh sách

- **Khởi tạo** – danh sách rỗng:

```
void SLL_Initialize(SLL_Type &First)
{  First = NULL;  }
```

- **Tạo nút mới** có dữ liệu NewData (trả về NULL nếu không đủ bộ nhớ):

```
SLL_Type SLL_Create_Node(T NewData)
{  SLL_Type Pnode = new SLL_OneNode;
   if (Pnode != NULL)
   {  Pnode->NextNode = NULL;
      Pnode->Key = NewData;
   }
   return (Pnode);
}
```

- **Duyệt danh sách** – khung cho mọi thao tác đi qua từng nút:

```
void SLL_Show(SLL_Type SList)
{  SLL_Type CurNode = SList;
   while (CurNode != NULL)
   {  printf("%4d", CurNode->Key);
      CurNode = CurNode->NextNode;   // sang nút kế
   }
   printf("\n");
}
```

Ví dụ 7.2 – Đếm số nút

```
int SLL_Count(SLL_Type SList)
{  int n = 0;
   for (SLL_Type p = SList; p != NULL; p = p->NextNode)
      n++;
   return n;
}
```

- **Không** dùng chính SList để duyệt (SList = SList->NextNode) ⇒ sẽ **mất** địa chỉ nút đầu!

2.2 Tìm kiếm trên danh sách liên kết đơn

- Chỉ áp dụng được **tìm tuyến tính** (không truy cập ngẫu nhiên). Hàm trả về **địa chỉ** nút tìm thấy (hoặc NULL) – thuận tiện cho thao tác thêm/hủy sau đó.

```
B1: CurNode = SList
B2: IF (CurNode = NULL OR CurNode->Key = SearchData)
      Thực hiện Bkt
B3: CurNode = CurNode->NextNode
B4: Lặp lại B2
Bkt: Kết thúc   // CurNode = NULL: không tìm thấy
```

```
SLL_Type SLL_Search(SLL_Type SList, T SearchData)
{  SLL_Type CurNode = SList;
   while (CurNode != NULL &&
          CurNode->Key != SearchData)
      CurNode = CurNode->NextNode;
   return (CurNode);
}
```

- Thứ tự điều kiện quan trọng: kiểm tra `CurNode != NULL` **trước** khi truy cập CurNode->Key.

Ví dụ 7.3 – Tìm 18 trong SLList = (10, 20, 18, 40)

Bước	CurNode trỏ tới	CurNode->Key	Hành động
1	nút 1	10	≠ 18 ⇒ sang nút kế
2	nút 2	20	≠ 18 ⇒ sang nút kế
3	nút 3	18	= 18 ⇒ **dừng**, trả về địa chỉ nút 3

- Tìm 77: CurNode đi qua 4 nút rồi bằng NULL ⇒ trả về **NULL**.
- **Phân tích:** O(N); có thể áp dụng **phần tử cầm canh** (thêm nút cuối chứa X) như buổi 2.

2.3 Thêm phần tử vào đầu danh sách

```
B1: NewNode = SLL_Create_Node(NewData)
B2: IF (NewNode = NULL) Thực hiện Bkt
B3: NewNode->NextNode = SLList  // nối SLList vào sau NewNode
B4: SLList = NewNode            // NewNode đứng đầu
Bkt: Kết thúc
```

```
SLL_Type SLL_Add_First(SLL_Type &SList, T NewData)
{  SLL_Type NewNode = SLL_Create_Node(NewData);
   if (NewNode == NULL)
      return (NULL);
   NewNode->NextNode = SList;
   SList = NewNode;
   return (SList);
}
```

- **Độ phức tạp O(1)**; đúng cả khi danh sách **rỗng** (SList = NULL ⇒ NewNode->NextNode = NULL).
- Tham số `SLL_Type &SList` (tham chiếu) vì hàm **thay đổi** con trỏ đầu.

Ví dụ 7.4 – Thêm 25 vào đầu danh sách (10, 20, 18, 40, 35, 30)

```
NewNode ●→[25|NULL]
SLList  ●→[10|●]→[20|●]→[18|●]→[40|●]→[35|●]→[30|NULL]

B3: NewNode->NextNode = SLList
NewNode ●→[25|●]┐
SLList  ●────────┴→[10|●]→[20|●]→ … →[30|NULL]

B4: SLList = NewNode
SLList  ●→[25|●]→[10|●]→[20|●]→[18|●]→[40|●]→[35|
●]→[30|NULL]
```

Sai thứ tự B3, B4

Nếu làm B4 trước (SLList = NewNode) thì địa chỉ nút 10 bị **mất** ⇒ cả danh sách cũ bị "thất lạc" (rò rỉ bộ nhớ). Nguyên tắc: **nối nút mới vào danh sách trước, rồi mới đổi con trỏ cũ**.

2.4 Thêm phần tử vào cuối danh sách

```
B1: NewNode = SLL_Create_Node(NewData)
B2: IF (NewNode = NULL) Thực hiện Bkt
B3: IF (SLList = NULL)
      B3.1: SLList = NewNode
      B3.2: Thực hiện Bkt
// Tìm đến địa chỉ của phần tử cuối cùng
B4: CurNode = SLList
B5: IF (CurNode->NextNode = NULL) Thực hiện B8
B6: CurNode = CurNode->NextNode
B7: Lặp lại B5
B8: CurNode->NextNode = NewNode  // nối vào sau nút cuối
Bkt: Kết thúc
```

```
SLL_Type SLL_Add_Last(SLL_Type &SList, T NewData)
{  SLL_Type NewNode = SLL_Create_Node(NewData);
   if (NewNode == NULL)
      return (NULL);
   if (SList == NULL)
   {  SList = NewNode;
      return (SList);
   }
   SLL_Type CurNode = SList;
   while (CurNode->NextNode != NULL)
      CurNode = CurNode->NextNode;
   CurNode->NextNode = NewNode;
   return (SList);
}
```

Ví dụ 7.5 – Thêm 25 vào cuối (15, 10, 20, 18, 40, 35)

CurNode đi từ 15 đến 35 (nút có NextNode = NULL) ⇒ 35->NextNode = NewNode ⇒ (15, 10, 20, 18, 40, 35, **25**).

- **Phân tích:** phải duyệt đến nút cuối ⇒ **O(N)**. Tạo danh sách N phần tử bằng cách thêm cuối liên tục tốn $O(N^2)$ ⇒ khắc phục bằng con trỏ **Last** (trang 3.2) để thêm cuối **O(1)**.

2.5 Thêm phần tử vào sau một nút và thêm có thứ tự

- **Thêm NewData vào ngay sau nút InsNode** (đã biết địa chỉ, ví dụ nhờ SLL_Search):

```
B1: NewNode = SLL_Create_Node(NewData)
B2: IF (NewNode = NULL) Thực hiện Bkt
// Nối các nút kế sau InsNode vào sau NewNode
B3: NewNode->NextNode = InsNode->NextNode
// Chuyển liên kết của InsNode về NewNode
B4: InsNode->NextNode = NewNode
Bkt: Kết thúc
```

```
SLL_Type SLL_Add_Mid(SLL_Type &SList, T NewData,
                     SLL_Type InsNode)
{  SLL_Type NewNode = SLL_Create_Node(NewData);
   if (NewNode == NULL) return (NULL);
   NewNode->NextNode = InsNode->NextNode;
   InsNode->NextNode = NewNode;
   return (SList);
}
```

```
Trước:  [15]→[10]→[20]→[18]→…     InsNode = nút 20
B3:     [25]──────────┐
        [15]→[10]→[20]→[18]→…
B4:     [15]→[10]→[20]→[25]→[18]→…
```

- **Thêm vào danh sách có thứ tự tăng** – tìm nút **cuối cùng có Key ≤ X** rồi thêm sau nó:

```
void SLL_Add_Order(SLL_Type &SList, T X)
{  if (SList == NULL || X < SList->Key)
   {  SLL_Add_First(SList, X);  return;  }
   SLL_Type CurNode = SList;
   while (CurNode->NextNode != NULL &&
          CurNode->NextNode->Key <= X)
      CurNode = CurNode->NextNode;
   SLL_Add_Mid(SList, X, CurNode);
}
```

Ví dụ 7.6

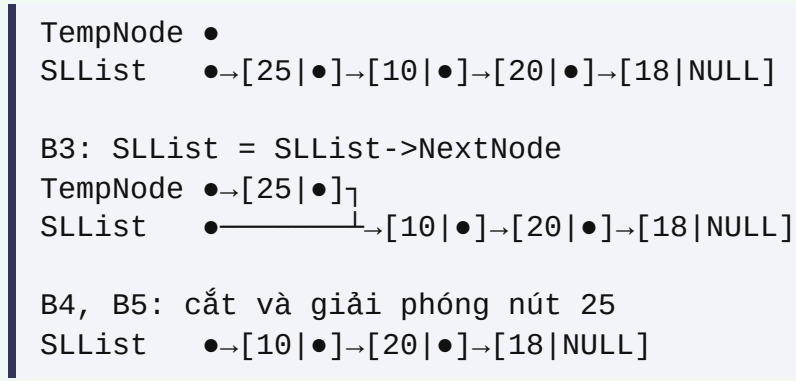
Lần lượt thêm 30, 12, 45, 12, 27 vào danh sách rỗng ⇒ (30) → (12, 30) → (12, 30, 45) → (12, 12, 30, 45) → **(12, 12, 27, 30, 45)**. Không cần **dời** phần tử nào như danh sách đặc – chỉ đổi 2 con trỏ.

2.6 Hủy phần tử đầu danh sách

```
B1: IF (SLList = NULL) Thực hiện Bkt
B2: TempNode = SLList          // giữ địa chỉ nút cần hủy
B3: SLList = SLList->NextNode  // nút kế lên làm đầu
B4: TempNode->NextNode = NULL  // cắt liên kết
B5: delete TempNode            // giải phóng bộ nhớ
Bkt: Kết thúc
```

```
int SLL_Delete_First(SLL_Type &SList)
{  if (SList == NULL)
      return (0);
   SLL_Type TempNode = SList;
   SList = SList->NextNode;
   TempNode->NextNode = NULL;
   delete TempNode;
   return (1);
}
```

Ví dụ 7.7 – Hủy nút đầu của (25, 10, 20, 18)



- **O(1)**. Nếu quên `delete` ⇒ **rò rỉ bộ nhớ**; nếu `delete SList` trước khi lấy SList->NextNode ⇒ truy cập vùng nhớ đã giải phóng (lỗi nghiêm trọng).

2.7 Hủy phần tử sau một nút và hủy theo khóa

- **Hủy nút đứng sau nút PreDelNode:**

```
int SLL_Delete_After(SLL_Type PreDelNode)
{  SLL_Type DelNode = PreDelNode->NextNode;
   if (DelNode == NULL)
      return (0);                  // không có nút sau
   PreDelNode->NextNode = DelNode->NextNode;
   DelNode->NextNode = NULL;
   delete DelNode;
   return (1);
}
```

```
Trước:  [10]→[20]→[18]→[40]      PreDelNode = 20
Sau:    [10]→[20]————→[40]       (nút 18 bị giải phóng)
```

- Vì sao cần nút **đứng trước**? DSLK đơn chỉ đi được một chiều; muốn "nối tắt" qua nút bị hủy phải sửa NextNode của nút trước nó.

- **Hủy nút có Key = DelData:** vừa tìm vừa ghi nhớ nút đứng trước.

```
int SLL_Delete_Node(SLL_Type &SList, T DelData)
{  SLL_Type DelNode = SList, PreDelNode = NULL;
   while (DelNode != NULL &&
          DelNode->Key != DelData)
   {  PreDelNode = DelNode;
      DelNode = DelNode->NextNode;
   }
   if (DelNode == NULL)
      return (0);                  // không tìm thấy
   if (PreDelNode == NULL)         // là nút đầu
      return SLL_Delete_First(SList);
   return SLL_Delete_After(PreDelNode);
}
```

Ví dụ 7.8 – Từ (25, 10, 20, 99, 18, 40, 35, 30)

Hủy 99 (PreDelNode = 20), hủy 25 (nút đầu), hủy 30 (nút cuối, PreDelNode = 35) ⇒ **(10, 20, 18, 40, 35)**; hủy 77 ⇒ trả về 0.

2.8 Hủy toàn bộ danh sách

- Khi không dùng danh sách nữa phải **giải phóng từng nút**; chỉ gán SLList = NULL sẽ làm **mất** toàn bộ vùng nhớ (rò rỉ).

```
B1: IF (SLList = NULL) Thực hiện Bkt
B2: TempNode = SLList
B3: SLList = SLList->NextNode
B4: delete TempNode
B5: Lặp lại B1
Bkt: Kết thúc
```

```
void SLL_Delete(SLL_Type &SList)
{  while (SList != NULL)
      SLL_Delete_First(SList);
}
```

- Sau khi hủy: SList = NULL ⇒ danh sách trở về trạng thái **khởi tạo**, dùng lại được.

Ví dụ 7.9 – Đoạn mã SAI thường gặp

```
for (SLL_Type p = SList; p != NULL; p = p->NextNode)
   delete p;      // SAI: p->NextNode đọc sau khi p đã bị
delete
```

- **Sửa:**

```
SLL_Type p = SList;
while (p != NULL)
{  SLL_Type q = p->NextNode;  // lưu nút kế TRƯỚC
   delete p;
   p = q;
}
SList = NULL;
```

Kiểm tra rò rỉ bộ nhớ

Biên dịch với `-fsanitize=address` (g++) hoặc dùng Valgrind / Visual Studio Diagnostic Tools – công cụ sẽ báo số byte chưa được giải phóng khi chương trình kết thúc.

2.9 Sắp xếp trên danh sách liên kết đơn

- Hai cách: **hoán vị dữ liệu** (Key) giữa các nút, hoặc **thay đổi liên kết** (di chuyển cả nút).
- **Cách 1 – Selection Sort hoán vị Key:**

```
void SLL_Selection_Sort(SLL_Type SList)
{  for (SLL_Type Ki = SList;
        Ki != NULL && Ki->NextNode != NULL;
        Ki = Ki->NextNode)
   {  SLL_Type MinNode = Ki;
      for (SLL_Type Kj = Ki->NextNode; Kj != NULL;
           Kj = Kj->NextNode)
         if (Kj->Key < MinNode->Key)
            MinNode = Kj;
      T Temp = Ki->Key;              // hoán vị dữ liệu
      Ki->Key = MinNode->Key;
      MinNode->Key = Temp;
   }
}
```

- Đơn giản, nhưng khi mỗi nút chứa **nhiều trường** (struct lớn) thì hoán vị dữ liệu tốn kém.

- **Cách 2 – Insertion Sort đổi liên kết:** lần lượt tách từng nút khỏi danh sách cũ, chèn vào đúng vị trí trong danh sách kết quả (không dời, không sao chép dữ liệu).

```
void SLL_Insertion_Sort(SLL_Type &SList)
{  SLL_Type Result = NULL;
   while (SList != NULL)
   {  SLL_Type Node = SList;          // tách nút đầu
      SList = SList->NextNode;
      if (Result == NULL || Node->Key < Result->Key)
      {  Node->NextNode = Result;  Result = Node;  }
      else
      {  SLL_Type Cur = Result;
         while (Cur->NextNode != NULL &&
                Cur->NextNode->Key <= Node->Key)
            Cur = Cur->NextNode;
         Node->NextNode = Cur->NextNode;
         Cur->NextNode = Node;
      }
   }
   SList = Result;
}
```

- (10, 20, 18, 40, 35) ⇒ **(10, 18, 20, 35, 40)**. Ổn định; MergeSort cũng rất phù hợp với DSLK (truy cập tuần tự).

2.10 Ví dụ: đảo ngược danh sách liên kết đơn

- **Bài toán:** đảo ngược thứ tự các nút **tại chỗ** (không tạo nút mới), chỉ bằng cách đổi hướng các con trỏ NextNode.

```
void SLL_Reverse(SLL_Type &SList)
{  SLL_Type Pre = NULL, Cur = SList;
   while (Cur != NULL)
   {  SLL_Type Next = Cur->NextNode;  // 1. lưu nút kế
      Cur->NextNode = Pre;            // 2. đảo liên kết
      Pre = Cur;                      // 3. tiến Pre
      Cur = Next;                     // 4. tiến Cur
   }
   SList = Pre;                       // nút cuối cũ thành
đầu
}
```

- Độ phức tạp **O(N)**, bộ nhớ phụ O(1) (3 con trỏ).

Ví dụ 7.10 – Đảo (10, 20, 30)

Vòng	Pre	Cur	Next	Liên kết sau vòng
Đầu	NULL	10	–	10 → 20 → 30 → NULL
1	10	20	20	NULL ← 10 20 → 30 → NULL
2	20	30	30	NULL ← 10 ← 20 30 → NULL
3	30	NULL	NULL	NULL ← 10 ← 20 ← 30

- SList = Pre = nút 30 ⇒ **(30, 20, 10)**.

- **Lỗi thường gặp:** quên bước 1 (lưu Next) ⇒ sau bước 2 mất đường đi tới phần còn lại của danh sách.

3.1 Các lỗi con trỏ thường gặp

Lỗi	Ví dụ	Hậu quả	Cách tránh
Truy cập con trỏ NULL	`p->Key` khi p = NULL	Chương trình bị dừng (crash)	Luôn kiểm tra `p != NULL` trước
Mất đầu danh sách	Duyệt bằng `SList = SList->NextNode`	Không truy cập lại được danh sách	Dùng biến phụ CurNode
Rò rỉ bộ nhớ	Gán `SList = NULL` mà không delete	Tốn RAM dần, chương trình chậm/treo	Hủy từng nút
Con trỏ treo (dangling)	Dùng p sau `delete p`	Kết quả ngẫu nhiên, crash	Gán `p = NULL` sau delete; lưu Next trước
Sai thứ tự gán liên kết	Đổi InsNode->NextNode trước khi nối NewNode	Mất phần sau danh sách	Nối nút mới trước, cắt cũ sau
Quên tham chiếu `&`	`void Add_First(SLL_Type SList, …)`	Thay đổi không có tác dụng ra ngoài hàm	Dùng `SLL_Type &SList`
Quên xử lý danh sách rỗng	Add_Last truy cập `SList->NextNode` khi SList = NULL	Crash	Kiểm tra trường hợp rỗng riêng

Ví dụ 7.11 – Quy trình kiểm thử một hàm DSLK

Luôn chạy thử với 4 trường hợp: danh sách **rỗng**, có **1 nút**, thao tác ở **đầu**, thao tác ở **cuối** – phần lớn lỗi con trỏ xuất hiện ở các trường hợp biên này.

3.2 Quản lý danh sách bằng hai con trỏ First và Last

- Lưu thêm con trỏ **Last** trỏ tới nút cuối ⇒ **thêm cuối O(1)**; nút cuối truy cập tức thì.

```
typedef struct {
   SLL_Type First;
   SLL_Type Last;
} SLL_List;
void Init(SLL_List &L) { L.First = L.Last = NULL; }
void AddLast(SLL_List &L, T X)
{  SLL_Type p = SLL_Create_Node(X);
   if (p == NULL) return;
   if (L.First == NULL)        // danh sách rỗng
      L.First = L.Last = p;
   else
   {  L.Last->NextNode = p;
      L.Last = p;
   }
}
void AddFirst(SLL_List &L, T X)
{  SLL_Type p = SLL_Create_Node(X);
   if (p == NULL) return;
   p->NextNode = L.First;
   L.First = p;
   if (L.Last == NULL) L.Last = p;   // trước đó rỗng
}
```

- **Lưu ý khi hủy:** nếu hủy nút **cuối** phải cập nhật Last; nếu danh sách trở nên rỗng phải gán **First = Last = NULL**.

```
int DeleteFirst(SLL_List &L)
{  if (L.First == NULL) return 0;
   SLL_Type p = L.First;
   L.First = p->NextNode;
   if (L.First == NULL) L.Last = NULL;
   delete p;
   return 1;
}
```

- Hủy nút cuối vẫn **O(N)** vì phải tìm nút đứng trước Last ⇒ khắc phục bằng **DSLK kép** (buổi 8).

Ví dụ 7.12 – Hiệu quả

Tạo danh sách 10^5 phần tử bằng thêm cuối: không có Last ≈ $5 \cdot 10^9$ bước duyệt ($N^2/2$); có Last chỉ **10^5** bước.

4.1 Bài thực hành buổi 7 – Quản lý danh sách sinh viên bằng DSLK đơn

Yêu cầu

```
typedef struct { char MaSV[11]; char HoTen[31]; float DTB; } SinhVien;
typedef struct SV_Node { SinhVien Data; SV_Node *NextNode; } SV_OneNode;
typedef SV_OneNode *SV_Type;
```

- 1) Nhập danh sách SV (thêm cuối, dùng con trỏ Last); xuất danh sách dạng bảng.
- 2) Tìm SV theo MaSV (trả về địa chỉ nút); sửa DTB của SV tìm được.
- 3) Thêm một SV vào **sau** SV có mã cho trước; thêm SV vào danh sách **có thứ tự** theo MaSV.
- 4) Xóa SV theo MaSV; xóa **tất cả** SV có DTB < 5 (chú ý trường hợp nhiều nút liên tiếp và nút đầu).
- 5) Sắp xếp theo DTB giảm dần bằng **đổi liên kết**; in 3 SV điểm cao nhất.
- 6) Tách danh sách thành 2 danh sách: "Đạt" (DTB ≥ 5) và "Không đạt" mà **không** tạo nút mới.
- 7) Hủy toàn bộ danh sách trước khi thoát chương trình; kiểm tra rò rỉ bằng `-fsanitize=address`.

4.2 Câu hỏi và bài tập

Câu hỏi

- 1) So sánh danh sách đặc và DSLK đơn về bộ nhớ, tốc độ truy cập, thêm, hủy.
- 2) Vì sao hàm thêm đầu dùng tham số `SLL_Type &SList` còn hàm duyệt thì không cần?
- 3) Vì sao không áp dụng hiệu quả tìm nhị phân trên DSLK đơn?
- 4) Muốn hủy một nút khi chỉ biết địa chỉ của chính nó (không biết nút trước) thì làm thế nào? (Gợi ý: chép dữ liệu nút sau vào rồi hủy nút sau – không áp dụng được cho nút cuối.)

Bài tập (có đáp số)

- 1) Cho SLList = (5, 12, 7, 12, 9). Thực hiện liên tiếp: thêm 3 vào đầu; thêm 20 vào sau nút có Key = 7; hủy nút đầu; hủy nút có Key = 12 (lần xuất hiện đầu). → **(5, 7, 20, 12, 9)**.
- 2) Viết hàm trả về **nút giữa** của danh sách chỉ với **một lần duyệt** (gợi ý: hai con trỏ, một con đi 1 bước, một con đi 2 bước).
- 3) Viết hàm **trộn** hai DSLK đơn có thứ tự thành một DSLK có thứ tự bằng cách đổi liên kết (không tạo nút mới).
- 4) Biểu diễn đa thức bằng DSLK đơn (mỗi nút: hệ số, số mũ); viết hàm cộng, nhân hai đa thức.